

CS 442: Lab 4 Report

Sonny Ngo, CS 442, smngo@crimson.ua.edu

Abstract—This report documents a comprehensive exploration of HTTPS and Public Key Infrastructure (PKI) concepts through a simulated Man-in-the-Middle (MITM) attack environment. Over six tasks, the project covered the generation and validation of SSL/TLS certificates, the configuration of Apache web servers with custom certificates, and the simulation of DNS spoofing to intercept and manipulate secure communications. Tools such as curl and openssl were used to validate certificate behavior under various scenarios, including trusted connections, certificate-domain mismatches, and untrusted certificate authorities. The cleanup phase ensured the dismantling of the MITM environment by disabling the fake site, removing DNS spoofing entries, and restoring the legitimate domain's functionality. The results highlight the critical role of PKI and HTTPS in ensuring secure communication and preventing attacks, emphasizing the importance of proper certificate management and DNS security practices. Screenshots and outputs accompany each step to demonstrate the process and provide a complete understanding of the system's behavior under attack and during recovery.

I. INTRODUCTION/LAB DEFINITION

The purpose was to explore the practical implementation of HTTPS and Public Key Infrastructure (PKI) to understand their role in ensuring secure communication and mitigating Man-in-the-Middle (MITM) attacks. The lab was divided into six tasks that progressively guided the creation, validation, and manipulation of SSL/TLS certificates, along with the configuration of a web server to simulate a secure and compromised environment.

The first set of tasks focused on generating a custom Certificate Authority (CA) and issuing certificates for a fake website (www.example.com) to simulate how attackers could exploit weaknesses in certificate management. Using Apache as the web server, configurations were implemented to enable HTTPS for the fake website, and DNS spoofing was employed to redirect traffic to this malicious server.

Tools like curl and openssl were used to analyze the behavior of HTTPS clients when encountering valid, invalid, and mismatched certificates. These tools provided insights into the verification process and highlighted the importance of matching domain names and trusted certificate chains.

The final parts involved the cleanup and validation of the environment, including disabling the fake site, removing DNS spoofing entries, and restoring the legitimate domain's functionality. This process emphasized the importance of

proper certificate management and secure DNS configurations in preventing and mitigating MITM attacks.

II. LAB SETUP WITH SCREENSHOTS/EXPLANATIONS

A. Task 1: Becoming a Certificate Authority

In this task, I created a root Certificate Authority (CA) to sign server certificates. This involved generating a private key for the CA, creating a self-signed certificate, generating a Certificate Signing Request (CSR) for the web server, and signing the CSR with the CA. The steps demonstrate how Public Key Infrastructure (PKI) works to establish trust for secure communications.

The purpose of this task was to gain hands-on experience with PKI by setting up a CA, generating server certificates, and exploring the trust relationship established between a CA and its signed entities. The task also highlights the role of digital certificates in preventing man-in-the-middle (MITM) attacks. I navigated to the lab setup folder containing the necessary Docker files:

```
[12/07/24]seed@VR:~$ cd Desktop
[12/07/24]seed@VR:~/Desktop$ cd LabSetup4
[12/07/24]seed@VR:~/Desktop/LabSetup4$ cd LabSetup
[12/07/24]seed@VR:~/Desktop/LabSetup4$ docker-compose build
Building web-server
Step 1/7 : FROM handsonsecurity/seed-server:apache-php
--> 2385d8e3da9f
Step 2/7 : ARG MM01R=/var/www/bank32
--> Running in c48dac807c80
--> Removed intermediate container c48dac807c80
--> d206da30ec4
Step 3/7 : COPY ./index.html ./index_red.html $MM01R/
--> d34fb41c7c5b
Step 4/7 : COPY ./bank32_apache_ssl.conf /etc/apache2/sites-available
--> 7b4fd75234eb
Step 5/7 : COPY ./certs/bank32.crt ./certs/bank32.key /certs/
--> ff996cadf67a
Step 6/7 : RUN chmod 400 /certs/bank32.key && chmod 644 $MM01R/index.html && chmod 644 $MM01R/index_red.html && a2ensite ba
nk32_apache_ssl
--> Running in e5ab6bf39bf9
Enabling site bank32_apache_ssl.
To activate the new configuration, you need to run:
service apache2 reload
--> Removed intermediate container e5ab6bf39bf9
--> f96d2907c531
Step 7/7 : CMD tail -f /dev/null
--> Running in d2ef91b17266
--> Removed intermediate container d2ef91b17266
--> 98d34ee3f5bc
Successfully built 98d34ee3f5bc
Successfully tagged seed-image-www-pki:latest
```

To act as the CA, I generated a private key for modelCA. This key is critical for signing certificates and must remain secure. I used the following command: `openssl genrsa -out modelCA.key 4096`. I verified the key's validity with: `openssl rsa -in modelCA.key -check`.

```

04:02:dd:04:f7:c2:00:00
[12/07/24]seed@VM:~/.../certs openssl x509 -req -in bank32.crt -CA modelCA.crt -CAkey modelCA.key -CAcreateserial V
> -out bank32.crt.days 365 -sha256
Certificate ok
subject=CN = www.bank32.com, o = Bank32 Inc., C = US
Getting CA Private Key
[12/07/24]seed@VM:~/.../certs openssl x509 -in bank32.crt -text -noout
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            1b:07:98:f4:90:3d:a9:42:2f:d9:38:0d:c1:25:02:b4:71:d6:1a:66
        Signature Algorithm: sha256WithRSAREncryption
        Issuer: CN = www.modelCA.com, o = Model CA LTD., C = US
        Validity
            Not Before: Dec 7 19:50:43 2024 GMT
            Not After : Dec 7 19:50:43 2025 GMT
        Subject: CN = www.bank32.com, o = Bank32 Inc., C = US
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            RSA Public-Key: (2048 bit)
            Modulus:
                00:07:8d:ca:f2:48:6a:bf:55:52:0b:c2:92:5e:0e:
                5b:b5:ac:f7:4e:26:6a:b0:3b:61:b4:94:94:38:f1:
                ed:d8:28:85:65:74:aa:d3:8f:06:07:b4:7a:5e:ce:8b:
                4b:ef:5b:52:c2:86:ee:27:61:bf:ba:e0:7c:74:6f:
                45:f3:9c:07:e0:ce:1c:11:71:0e:06:b3:95:41:df:
                7b:fd:19:f2:b1:30:26:20:ef:0f:80:fe:60:07:9e:
                be:10:6e:ff:ca:7b:5:9d:4e:5b:18:66:9c:11:c9:
                d5:ba:67:ba:5e:19:08:b0:97:cd:71:3f:d8:66:82:
                8a:c5:07:5b:da:4e:1b:aab:fa:6b:36:aa:34:5f:
                fe:f4:a4:7a:cf:07:55:f0:63:65:4a:c3:97:4a:1f:
                8e:bd:00:25:50:8d:bc:5e:e0:82:4e:25:78:27:7f:
                ec:0b:fd:66:0d:8f:42:04:5d:91:00:9f:be:77:f9:
                e2:96:28:1d:f9:fa:98:5a:5a:9f:b0:6c:2f:8f:3c:
                c3:29:aa:c5:5a:64:9c:f3:1a:06:05:08:b0:bd:8e:
                b6:c2:9e:0e:cf:97:33:04:c1:82:2d:fe:2a:5d:55:
                d4:2d:04:49:a5:08:82:c1:05:25:2e:87:4d:1a:
                fe:86:57:77:08:12:29:1e:be:73:d5:f6:2a:04:5a:
                04:dd
            Exponent: 65537 (0x10001)
        Signature Algorithm: sha256WithRSAREncryption
            a4:35:dd:92:05:04:23:53:15:50:13:0b:1d:8f:89:0e:f5:12:2:
            31:6e:7b:85:b7:90:c7:f3:ae:03:b8:b2:55:3a:0e:84:22:65:
            1a:fb:5f:ed:0d:b6:ce:0a:7a:d3:07:87:07:b7:24:8e:03:dd:
            ff:a4:31:38:74:a2:8b:76:00:90:54:96:41:37:02:f8:da:7a:
            5f:2f:e2:fa:02:18:ea:7f:09:92:0c:a5:14:d8:64:69:4e:
            ab:c3:41:04:15:a1:81:93:1f:4d:49:73:22:9c:2b:cd:dd:3b:
            7c:04:61:39:41:f0:de:64:32:1f:72:f7:aea:37:98:58:b9:92:
            9c:f3:05:12:ac:09:29:c6:bb:20:ec:24:24:10:10:12:c:81:
            23:e1:ab:28:3f:ff:fe:31:90:01:92:02:46:08:c9:06:ae:22:
            24:e7:af:07:32:92:f7:b6:c2:61:59:0f:e1:7f:24:92:06:55:
            9e:da:ab:b6:c1:9f:31:19:37:c2:14:93:9c:2e:dd:7a:72:
            73:20:19:05:17:bd:df:b5:9e:95:34:fed:b7:73:74:7c:1d:
            9f:eb:08:32:fb:0e:6a:17:7f:12:67:30:5a:0f:71:73:2a:4a:
            06:53:08:09:42:4:ef:ef:ba:7c:ca:4b:30:8f:c1:e4:3a:57:
            0f:eb:dc:73:61:50:00:e0:12:13:35:7d:ee:0d:81:1b:20:0e:43:
            3c:03:51:56:3c:19:33:3c:4b:5c:0f:01:1d:e2:7c:01:b6:
            88:e9:5f:16:12:ab:1b:0e:14:6b:11:eb:b8:08:62:62:52:
            df:cd:b1:a9:57:42:06:25:b3:4d:99:9f:46:0b:c9:4:c41:df:
            1a:04:09:57:16:cd:09:b0:14:ff:ab:c2:c3:70:0f:01:21:
            40:aa:3b:66:0e:57:71:39:78:19:0f:a1:e6:4e:cc:91:1e:
            65:6e:84:24:eb:2d:0d:32:83:7e:14:88:95:0b:23:86:64:2:
            7b:3c:95:15:62:5e:1d:00:7f:18:90:43:7f:5b:3e:6e:28:b9:
            76:49:26:62:28:5b:53:44:19:3f:15:10:fca1:61:cd:
            f3:41:65:62:7b:c3:5f:74:be:3b:de:c0:3a:05:22:ea:ad:68:
            07:6b:c0:88:98:ca:e5:eb:cd:66:4e:c2:06:6e:11:30:4a:02:
            54:f5:6c:3f:26:3b:03:4a:dc:26:29:da:dc1:1c:6a:50:05:
            75:a9:a3:c4:b2:1d:7a:00:9e:fe:72:78:86:e1:7b:7c:0e:
            92:82:55:fc:1a:19:72:85:74:7d:5c:47:57:2a:0d:ba:3e:

```

To confirm that the server certificate matches the private key, I ran:

```
[12/07/24]seed@VM:~/.../certs$ openssl x509 -noout -modulus -in bank32.crt | openssl md5
(stdin)= 90de1e47bde1f446b508ffe22e742678
[12/07/24]seed@VM:~/.../certs$ openssl rsa -noout -modulus -in bank32.key | openssl md5
Enter pass phrase for bank32.key:
(stdin)= 90de1e47bde1f446b508ffe22e742678
[12/07/24]seed@VM:~/.../certs$
```

The outputs matched, confirming the private key and certificate were correctly paired. Potential issues during the task included ensuring the correct container setup and managing mismatched certificates and private keys, which were mitigated by careful verification at each step.

B. Task 2: Generating a Certificate Request for Your Web Server

In this task, I generated a Certificate Signing Request (CSR) for the web server `www.bank32.com`. This included creating a private key, adding Subject Alternative Names (SANs), and verifying the CSR's contents to ensure it met the lab requirements. These steps are critical for securing the web server using Public Key Infrastructure (PKI).

The goal of this task was to generate a CSR that contains the web server's public key and identity information, including alternative domain names. The CSR will later be signed by the Certificate Authority (CA) to generate a server certificate, which will be used for HTTPS communications.

Using OpenSSL, I generated the private key and CSR for the web server. The `bank32.key` file contains the private key for the server. The `bank32.csr` file is the Certificate Signing Request. The Subject Alternative Name (SAN) extension was added to include the server's additional domain names: `www.bank32A.com`, `www.bank32B.com`, and `www.bank32W.com`.

```
[12/07/24]seed@VM:~/.../certs$ openssl req -newkey rsa:2048 -keyout bank32.key -out bank32.csr \
> -subj "/CN=www.bank32.com/O=Bank32 Inc./C=US" \
> -passout pass:dees
Generating a RSA private key
.....++++
writing new private key to 'bank32.key'
.....
```

I verified the CSR to ensure the details were accurate and included the required fields. Subject: Common Name (CN): `www.bank32.com`, Organization (O): `Bank32 Inc.m`, Country (C): `US`, Extensions: `X509v3 Subject Alternative Name (SAN)` included: `DNS:www.bank32.com`, `DNS:www.bank32A.com`, `DNS:www.bank32B.com`, `DNS:www.bank32W.com`

```
[12/07/24]seed@VM:~/.../certs$ openssl req -newkey rsa:2048 -keyout bank32.key -out bank32.csr \
> -subj "/CN=www.bank32.com/O=Bank32 Inc./C=US" \
> -addext "subjectAltName=DNS:www.bank32.com,DNS:www.bank32A.com,DNS:www.bank32B.com,DNS:www.bank32W.com" \
> -passout pass:dees
Generating a RSA private key
.....++++
writing new private key to 'bank32.key'
```

```
[12/07/24]seed@VM:~/.../certs$ openssl req -in bank32.csr -text -noout
Certificate Request:
Data:
  Version: 1 (0x0)
  Subject: CN = www.bank32.com, O = Bank32 Inc., C = US
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public-Key: (2048 bit)
    Modulus:
      00:e3:fa:6e:d8:92:1d:7f:e6:90:fd:a6:de:df:5d:
      92:d2:39:f5:6a:c3:72:45:2f:d2:71:59:98:7b:44:
      2b:60:2c:8c:65:98:fa:5b:47:70:43:d8:4e:4e:6d:
      41:a7:ae:4f:13:2f:38:50:d5:36:f2:f5:15:c8:48:
      5b:0b:72:02:e6:29:4d:a8:76:54:cc:9f:59:73:b1:
      0d:2e:f8:ff:9f:67:5b:10:e4:c3:79:be:5e:c6:72:
      1c:5f:7f:47:a3:aa:35:51:d2:f6:73:a7:bb:20:d6:
      47:3f:ee:9a:47:41:b2:f8:0c:aa:37:26:1e:6f:29:
      cf:ac:22:e1:7c:7c:cb:ce:2b:57:5d:13:ea:a8:4c:
      d9:a1:b9:cd:60:18:c2:53:bb:1c:2e:8b:c9:a1:1f:
      25:9f:35:53:2c:88:a5:68:3e:89:f0:a9:71:21:a6:
      17:d4:a5:59:ef:5b:f4:2e:c2:3a:17:d5:d6:62:
      f1:4b:26:ee:8d:6d:47:25:00:89:0a:b0:53:7c:36:
      86:40:7a:4a:b8:69:64:47:c9:45:60:7d:60:5f:aa:
      0f:83:a1:a1:ab:2d:68:e8:90:df:15:06:69:78:53:
      af:df:49:35:60:10:03:bc:66:f8:00:4c:f5:bb:18:
      66:e8:61:c4:69:c8:fb:3b:d7:c6:f7:1f:c6:b3:a6:
      41:a7
    Exponent: 65537 (0x10001)
  Attributes:
  Requested Extensions:
    X509v3 Subject Alternative Name:
      DNS:www.bank32.com, DNS:www.bank32A.com, DNS:www.bank32B.com, DNS:www.bank32W.com
  Signature Algorithm: sha256WithRSAEncryption
  e0:49:1f:b1:32:0b:2c:b4:3b:9b:69:d8:3e:4a:5d:8b:b6:74:
  ea:49:fc:fc:6a:a3:e1:b0:72:c6:70:c6:db:aa:f7:ef:f3:6c:
  7d:03:c1:8f:9f:52:02:a6:8d:05:5c:29:ff:42:07:81:f1:c9:
  0d:48:a6:bf:88:5d:15:1d:3f:33:94:3f:99:ce:a1:94:98:20:
  f7:a9:42:32:21:b3:7b:6a:6c:a1:91:8e:73:94:4f:09:39:a3:
  58:44:14:3c:ab:bd:31:5c:d3:35:e3:d9:77:ef:dd:7b:57:05:
  0d:4a:86:27:7f:a7:5c:63:2e:26:e4:26:a9:d9:8a:8c:bb:7c:
  11:4f:69:9c:f7:8f:e7:90:07:9a:9c:9d:44:a9:96:97:61:cf:
  b1:9c:4a:de:22:1a:07:39:42:cf:75:d8:d9:07:92:3f:4f:
  ce:f0:da:71:27:e0:de:d8:a8:e9:04:76:48:2c:54:03:7e:08:
  d7:e9:0d:0d:15:c2:f7:b0:bd:a8:b8:df:35:cd:81:55:b4:ea:
  bc:84:9a:b0:bd:f0:8f:c9:17:f6:09:f2:a8:b0:8c:33:d0:6e:
  af:a2:a6:85:2c:54:88:eb:d3:ce:be:79:c3:e5:06:b0:5f:0e:
  a2:e0:85:69:f2:79:df:fa:ae:07:cd:33:04:aa:c0:df:b8:42:
  58:1d:d6:03
```

The generated files were saved in the `certs` directory: `bank32.key`: Private key for the web server. `bank32.csr`: CSR for the web server. The files are now ready for the next task, where the CSR will be signed by the CA to generate the server certificate.

C. Task 3: Generating a Certificate for your server

In this task, I used the Certificate Authority (CA) created in Task 1 to sign the Certificate Signing Request (CSR) generated in Task 2. This process resulted in the creation of the server certificate (`bank32.crt`) for `www.bank32.com`. The signed certificate will be used to secure the server in subsequent tasks. The purpose of this task was to generate a signed certificate for the server using the root CA. This signed certificate establishes the server's authenticity and enables secure communication over HTTPS. The process involved using the CA's private key and certificate to sign the server's CSR and verifying the resulting certificate.

I signed the `bank32.csr` using the root CA's private key and certificate to produce the signed server certificate `bank32.crt`.

```
[12/07/24]seed@VM:~/.../certs$ openssl x509 -req -in bank32.csr -CA modelCA.crt -CAkey modelCA.key -CAcreateserial \
> -out bank32.crt -days 365 -sha256
Signature ok
subject=CN = www.bank32.com, O = Bank32 Inc., C = US
Setting CA Private Key
[12/07/24]seed@VM:~/.../certs$
```

I then verified the signed certificate to ensure it was valid and correctly signed. I checked that `X509v3 Subject Alternative Name` included `www.bank32.com`, `www.bank32A.com`, `www.bank32B.com`, and `www.bank32W.com`.

```
[12/07/24]seed@VM:~/.../certs$ openssl x509 -in bank32.crt -text -noout
Certificate
Data:
  Version: 1 (0x0)
  Serial Number:
    1b:07:00:f4:00:3d:e9:42:2f:d9:30:0d:c1:25:02:b4:71:d6:1a:c
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: CN = www.modelCA.com, O = Model CA LTD., C = US
  Validity
    Not Before: Dec 7 21:14:32 2024 GMT
    Not After: Dec 7 21:14:32 2024 GMT
  Subject: CN = www.bank32.com, O = Bank32 Inc., C = US
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public-Key: (2048 bit)
    Modulus:
      80:ed:3:f4:6e:08:92:3d:7f:e6:90:f1:a6:de:0f:5d:
      03:43:~443/tcp,::443~443/tcp www-10.9.0.80
    Exponent: 65537 (0x10001)
  Signature Algorithm: sha256WithRSAEncryption
    30:06:7a:f1:76:a3:be:94:fd:09:38:49:2f:c3:b3:4c:08:06:
    7a:29:ec:b1:36:7e:60:36:2a:ba:a1:0d:29:57:65:07:00:16:
    fd:63:6a:63:aee:1a:01:40:04:ee:9a:bf:c9:c9:73:35:05:78:
    de:21:08:a1:8c:10:02:02:f0:0b:f8:b2:a1:64:c0:5f:31:ad:
    06:04:29:3a:57:0e:11:0a:60:08:03:90:0b:0f:07:02:
    9e:40:d5:a1:c3:ac:be:08:f9:f2:08:94:1a:10:53:05:0a:c1:
    52:62:00:5b:f9:17:31:1c:58:08:00:c1:17:54:04:1a:56:
    27:49:09:0f:06:18:a5:a8:c6:09:c2:a0:91:75:5c:6a:4a:26:
    09:08:5a:06:27:77:0f:c2:3a:9f:41:73:91:13:10:c3:4b:
    7:c4:c3:6f:23:a3:44:93:f4:05:07:10:06:27:52:09:97:47:
    f4:5a:b2:100:2a:f5:13:0a:c0:00:c7:0a:10:0a:0c:f0:3f:0b:
    c3:9a:c8:0a:f2:09:04:00:2c:3c:0f:f1:09:3c:02:30:02:6c:
    70:2b:0a:11:07:10b:2f:3a:10:09:9a:39:53:47:4e:4a:55:7a:
    05:19:11:12:7f:0b:07:5a:3a:0b:37:c2:94:02:0e:0c:f0:
    93:4a:04:98:00:07:5a:3a:0b:37:c2:94:02:0e:0c:f0:
    07:6c:23:64:c3:e4:03:c9:13:63:2a:26:1c:f4:0a:0b:
    c3:13:58:97:02:37:92:56:b2:be:c7:45:0e:12:04:11:c2:0f:
    76:4a:f0:30:5c:13:1a:00:93:0a:f5:00:c1:04:06:08:09:
    42:0d:0f:11:0b:03:1e:24:5e:3c:09:1f:be:0b:10:0f:73:0c:
    04:ee:c4:30:54:56:3c:06:79:09:31:0e:82:ee:35:03:0a:
    c9:08:12:0e:00:00:00:00:00:00:00:00:00:00:00:00:
    34:2f:c3:c9:b3:ec:09:c3:5c:2d:52:f8:06:1c:07:04:08:0c:
    62:7a:f9:07:7e:00:07:76:41:0a:03:71:b4:39:09:04:
    9a:82:c0:e4:09:07:c3:03:79:a9:30:15:05:40:170:76:17:
    25:07:73:43:41:0a:13:00:00:1a:fc:10:3c:02:04:f6:0c:bd:
    2b:04:08:0e:1f:5e:0a:0b:27:09:30:06:1a:3a:6f:2b:0c:cc:
    7a:1a:08:f3:10b:00:c4:0e:05:c0f:10b:c3:1ba:38:c7:55:00:c3:
    ad:07:10a:05:0e:f4:01:0e:00:00:00:00:00:00:00:00:
    af:09:26:f1:56:08b:3f:35
```

To confirm that the signed certificate matched the server’s private key, I ran:

```
12/07/24]seed@VM:~/.../certs$ openssl x509 -noout -modulus -in bank32.crt | openssl md5
(stdin)= ea2d18708413f9edfc344bdfc012426
12/07/24]seed@VM:~/.../certs$ openssl rsa -noout -modulus -in bank32.key | openssl md5
(stdin)= ea2d18708413f9edfc344bdfc012426
12/07/24]seed@VM:~/.../certs$ █
```

The MD5 hashes from both commands were identical, confirming the certificate and private key were correctly paired. This task showed the importance of using a trusted CA to issue server certificates. By signing the CSR with the CA, I made sure that the resulting certificate could be trusted by clients that recognize the CA. The verification process eliminated potential errors, such as mismatched certificates and private keys.

D. Task 4: Deploying Certificate in an Apache-Based HTTPS Website

I configured an HTTPS web server using Apache and deployed the signed certificate generated in Task 3. This involved copying the required files to the server, configuring Apache for HTTPS, and verifying the server’s functionality. Successful deployment enables secure communication over HTTPS. The objective of this task was to set up a web server with HTTPS using a certificate signed by a custom Certificate Authority (CA). The task demonstrated how to configure Apache for secure communication and verify the server’s HTTPS functionality.

I edited the .conf file with the command: nano bank32_apache_ssl.conf.

```
GNU nano 4.8 bank32_apache
VirtualHost *:443>
DocumentRoot /var/www/bank32
ServerName www.bank32.com
ServerAlias www.bank32A.com
ServerAlias www.bank32B.com
ServerAlias www.bank32W.com
DirectoryIndex index.html
SSLEngine On
SSLCertificateFile /certs/bank32.crt
SSLCertificateKeyFile /certs/bank32.key
</VirtualHost>

<VirtualHost *:80>
DocumentRoot /var/www/bank32
ServerName www.bank32.com
DirectoryIndex index_red.html
</VirtualHost>

# Set the following glocal entry to suppress an annoying warning message
ServerName localhost
```

The following files were copied into the Docker container:

bank32.crt: Signed server certificate, bank32.key: Server private key, modelCA.crt: Root CA certificate, bank32_apache_ssl.conf: Apache configuration file.

The files were copied using the following commands:

```
9e72f6727a0c seed-image-wws-pkl "/bin/sh -c 'tail -f.'" 19 minutes ago Up About a minute 0.0.0.0:80->80/tcp, ::80->80, 443->443/tcp, ::443->443/tcp www-10.9.0.80
[12/07/24]seed@VM:~/.../image_wws$ docker cp certs/bank32.crt www-10.9.0.80:/certs/
Successfully copied 3.07kB to www-10.9.0.80:/certs/
[12/07/24]seed@VM:~/.../image_wws$ docker cp certs/bank32.key www-10.9.0.80:/certs/
Successfully copied 3.58kB to www-10.9.0.80:/certs/
[12/07/24]seed@VM:~/.../image_wws$ docker cp certs/modelCA.crt www-10.9.0.80:/certs/
Successfully copied 3.58kB to www-10.9.0.80:/certs/
[12/07/24]seed@VM:~/.../image_wws$ docker cp bank32_apache_ssl.conf www-10.9.0.80:/etc/apache2/sites-available/bank32_ssl.conf
Successfully copied 2.56kB to www-10.9.0.80:/etc/apache2/sites-available/bank32_ssl.conf
[12/07/24]seed@VM:~/.../image_wws$ █
```

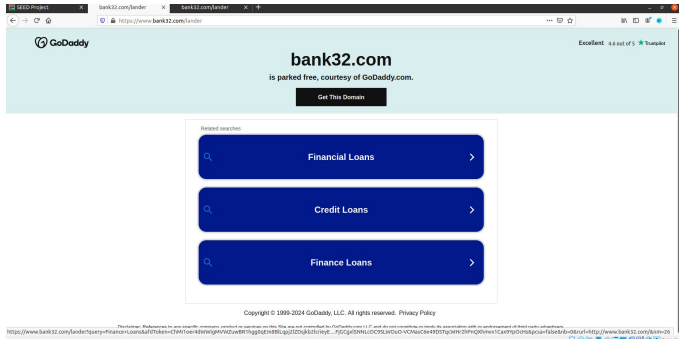
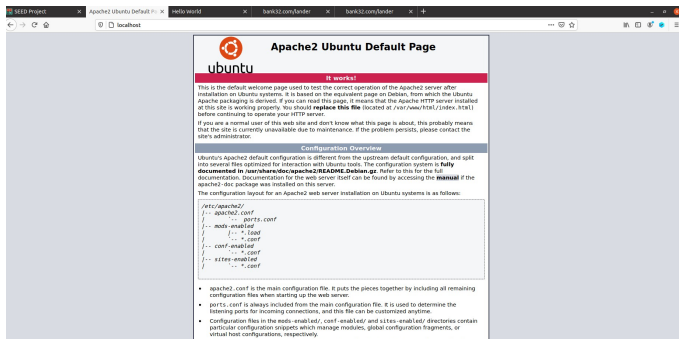
Inside the Docker container, I enabled SSL and the virtual host configuration using the following commands:

```
[12/07/24]seed@VM:~/.../image_wws$ docker exec -it www-10.9.0.80 /bin/bash
root@9e72f6727a0c:/# ls /certs
bank32.crt bank32.key modelCA.crt
root@9e72f6727a0c:/# ls /etc/apache2/sites-available
000-default.conf bank32_apache_ssl.conf bank32_ssl.conf default-ssl.conf
root@9e72f6727a0c:/# █
```

```
root@9e72f6727a0c:/# a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
root@9e72f6727a0c:/# a2ensite bank32_ssl.conf
Enabling site bank32_ssl.
To activate the new configuration, you need to run:
  service apache2 reload
root@9e72f6727a0c:/# service apache2 restart
* Restarting Apache httpd web server apache2
Enter passphrase for SSL/TLS keys for www.bank32.com:443 (RSA):
```

```
root@9e72f6727a0c:/#
```

I opened a browser and navigated to: HTTP: http://localhost and http://www.bank32.com, HTTPS: https://localhost and https://www.bank32.com, I observed: HTTP served index_red.html, HTTPS served index.html.




```

1 <VirtualHost *:443>
2     DocumentRoot /var/www/bank32
3     ServerName www.bank32.com
4     ServerAlias www.bank32a.com
5     ServerAlias www.bank32b.com
6     ServerAlias www.bank32w.com
7     DirectoryIndex index.html
8     SSLEngine On
9     SSLCertificateFile /certs/bank32.crt
10    SSLCertificateKeyFile /certs/bank32.key
11 </VirtualHost>
12
13 <VirtualHost *:80>
14     DocumentRoot /var/www/bank32
15     ServerName www.bank32.com
16     DirectoryIndex index_red.html
17 </VirtualHost>
18
19 <VirtualHost *:443>
20     DocumentRoot /var/www/bank32
21     ServerName www.example.com
22     DirectoryIndex index.html
23     SSLEngine On
24     SSLCertificateFile /certs/bank32.crt
25     SSLCertificateKeyFile /certs/bank32.key
26 </VirtualHost>
27
28 # Set the following global entry to suppress an annoying warning message
29 ServerName localhost

```

A screenshot of a web-based authentication dialog box. The dialog has a white background with a thin grey border. At the top, the title 'Authentication Required' is centered in a large, bold, black font. Below the title, a message in a smaller black font reads: 'Authentication is required to restart 'apache2.service''. In the center of the dialog is a black silhouette of a person's head and shoulders. Directly below the icon, the word 'SEED' is displayed in a bold, black, sans-serif font. At the bottom of the dialog is a password input field. It consists of a rectangular box with a thin red border. On the left side of the box are four black dots, and on the right side is a small, grey, circular icon with a diagonal slash. Below the dialog box, there are two buttons: 'Cancel' on the left and 'Authenticate' on the right, both in a black, sans-serif font.

```

GNU nano 4.8
127.0.0.1      localhost
127.0.1.1      VM

# The following lines are desirable for IPv6 capable hosts
::1    ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

# For DNS Rebinding Lab
192.168.60.80    www.seedIoT32.com

# For SQL Injection Lab
10.9.0.5          www.SeedLabSQLInjection.com

# For XSS Lab
10.9.0.5          www.xsslabelgg.com
10.9.0.5          www.example32a.com
10.9.0.5          www.example32b.com
10.9.0.5          www.example32c.com
10.9.0.5          www.example60.com
10.9.0.5          www.example70.com

# For CSRF Lab
10.9.0.5          www.csrflabelgg.com
10.9.0.5          www.csrflab-defense.com
10.9.0.105        www.csrflab-attacker.com

# For Shellshock Lab
10.9.0.80         www.seedlab-shellshock.com

# For MITM Lab
10.9.0.80         www.example.com

```

```
[12/07/24]seed@VM:~/.../image_www$ ping www.example.com
PING www.example.com (10.9.0.80) 56(84) bytes of data.
64 bytes from www.seedlab-shellshock.com (10.9.0.80): icmp_seq=1 ttl=64 time=0.053 ms
64 bytes from www.seedlab-shellshock.com (10.9.0.80): icmp_seq=2 ttl=64 time=0.050 ms
64 bytes from www.seedlab-shellshock.com (10.9.0.80): icmp_seq=3 ttl=64 time=0.060 ms
```

```
# For MITM Lab
10.9.0.80      www.fake-example.com
```

The screenshot shows a web browser window with a yellow warning banner at the top. The address bar displays 'https://www.fake-example.com'. The main content area features a yellow background with a warning icon (a yellow triangle with an exclamation mark) and the text 'Warning: Potential Security Risk Ahead'. Below this, a paragraph explains that a potential security threat was detected and the browser did not continue to the website. It warns that attackers could steal information like passwords, emails, or credit card details. A section titled 'What can you do about it?' follows, stating that the issue is most likely with the website and that there is nothing the user can do to resolve it. It suggests reaching out to the support team if on a corporate network or using antivirus software, and also recommends notifying the website's administrator. At the bottom of the warning area, there are two buttons: 'Go Back (Recommended)' and 'Advanced'.

```

12/07/24|seeds|mv-./.../certs$ openssl s_client -connect www.example.com:443 -ciphers modelCA.crt
CONNECTED(00000003)
depth=1 CN = www.modelCA.com, O = Model CA LTD., C = US
verify return=1
depth=0 CN = www.bank32.com, O = Bank32 Inc., C = US
verify return=1
---
Certificate chain
 0 s:CN = www.bank32.com, O = Bank32 Inc., C = US
 1:CN = www.modelCA.com, O = Model CA LTD., C = US
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIEA1CAEoCFBUhnp5QPAL9k4DcElARx1h8pMAGCSqG5G1b3QOEBCwIAMDBx
5DAWBgNvBAMMD3d3dy5tbZr1eBNLmNbvtEMMBOGA1IECgwNTW9kZWg0G9ETFE
LjLElMAKGA1UEBHMVMMhkcNMjQxMjA3MjExNDYwMjYwMjYwMjYwMjYwMjYw
RcVFOYDQVODDAs3d3cUyMvFuaZMYbTUEMB1GA1UECQw1MjYwMjYwMjYw
CzA3BgNvBAYTA1VTMlB1J1ANBgkqhkiG9w0BAQoFAAQAM0A8MIIBgcKAQEA4/pu
Z1JidH+0v/abe31250j1nashYRS/SCvmtE90rYCYMZJ76w0dw4Q9H0t1Bp65PEY84
JNU2vU9VYehb3C1C51lnqH2J9z9Zc7ENLjY/r4n0E0ZTDe5extnc3N0h66o1UdLw
c6e7JH2P4+sa80g+AyenYeybnPrCLh4fz1EX0RqfQezZobnWb1MY7U(CbU7s
p)R81nzVTL11la6eJ8K1xIAYXIKVZ71b0LS16t+hFVmlx5zXqWj1H0JC3C(RTFDdaG
dHpkUg1kR8F1lYX6oP6gghy1o6J3DFF0bpeF0v30k1YBADv64AE1rxhdmGHE
acj709fG9x/Gs6ZBpwIDAQABMAGCSqG5G1b3QOEBCwIAMDA1CAQAw1UdHxd0+LP03
D0kvz7NM4A2Z+KeyxPX5mN126od8pV2mHAB9Y2tjrh2hQNTwb/AyXMT1xjEidSh
jGwykvjW9rJAZM1fMaWGLCk6V57B6k1i1k1jKPCPh4KeQ0nH4ay+Pny3Z3qdHVPv
a+G1Y02FW/kfMRKZ1g3BFzS0GLVnsdkPBH1lqMYJqarDXvSua13CfvaJ33fHPvU6
l0xzkTG2wUjyHTNNvI6FEK/rVtxGLK1JL0f0wrkdkUvxi3d56a74zwzPjms1qj
Z1rUsW17YzPzrBAmx4K6oR17sv0NAH1jTR05XqXq1EFEC5SpzqfzRohNkxhdL
9VuTjldmAshk0r+1zF5L1LlUjPVXbCNw+TTC+4YtYmWFPZovR0E11X0jEsXBkx
x8bF6vYb5q9dggo5A2r3bAKVLrYjceVPMZM17ENTCgWb1v5+pxwplvtUvGH1d2
21xIevCXf0ndu4Gv5N/ujm0RMYags32b1hgnpmBXQV0B17fbc153ND0SAhMka
/rk8B0r2DL0R1LiEh1671yJMJYe0dm8rM50tj)xAOnESeXxmY+MMkV4M0t1B7qL
nv32PkrsEmi028bhcKXnW5a/Vgg/NQ=
-----END CERTIFICATE-----
subject=CN = www.bank32.com, O = Bank32 Inc., C = US

issuer=CN = www.modelCA.com, O = Model CA LTD., C = US
---
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
SSL handshake has read 1586 bytes and written 397 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated

```

DNS Spoofing and Certificate Validity: When accessing <https://www.example.com>, the browser trusted the connection because the certificate matched the domain and was signed by the trusted CA (modelCA). Domain Mismatch: Accessing <https://www.fake-example.com> triggered a browser warning because the certificate's Common Name (CN) did not match the requested domain. OpenSSL Validation: The certificate chain was successfully verified using `modelCA.crt`.

F. Task 6: Launching a Man-In-The-Middle Attack with a Compromised CA

The goal of this task was to clean up the simulated Man-in-the-Middle (MITM) attack environment and restore the system to its original state. This involved disabling the fake site configuration, removing DNS spoofing entries, and verifying that the fake server no longer intercepted traffic for the targeted domain. The task also ensured that the legitimate `www.example.com` could be accessed without interference.

The first step was to test HTTPS connections to `www.example.com` using the `curl` command. The following scenarios were tested: Ignoring Certificate Warnings: Using the `-k` flag, `curl` was instructed to bypass certificate validation:

```
12/07/24]seodotMT-./.../certs curl -k https://www.example.com
<!doctype html>
<html>
<head>
<title>Example Domain</title>

<meta charset="utf-8" />
<meta http-equiv="Content-type" content="text/html; charset=utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<style type="text/css">
body {
background-color: #f0f0f2;
margin: 0;
padding: 0;
font-family: .apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
}
div {
width: 600px;
margin: 5em auto;
padding: 2em;
background-color: #f0f0ff;
border-radius: 0.5em;
border: 1px solid #ccc;
box-shadow: 2px 2px 2px rgba(0,0,0,0.2);
}
a:link, a:visited {
color: #336699;
text-decoration: none;
}
@media (max-width: 780px) {
div {
margin: 0 auto;
width: auto;
}
}
</style>
</head>

<body>
<div>
<h1>Example Domain</h1>
<p>This domain is for use in illustrative examples in documents. You may use this
domain in literature without prior coordination or asking for permission.</p>
<p><a href="https://www.iana.org/domains/example">More information...</a></p>
</div>
</body>
</html>
```

This command successfully retrieved the content served by the fake server without validating the certificate.

```
[12/07/24]seede@VM:~/.../certs$ curl https://www.example.com --cacert modelCA.crt
<doctype html>
<html>
<head>
<title>Example Domain</title>

<meta charset="utf-8" />
<meta http-equiv="Content-type" content="text/html; charset=utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<style type="text/css">
body {
  background-color: #f0f0f2;
  margin: 0;
  padding: 0;
  font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
}
div {
  width: 600px;
  margin: 5em auto;
  padding: 2em;
  background-color: #f0f0ff;
  border-radius: 8.5em;
  box-shadow: 2px 3px 7px rgba(0,0,0,0.02);
}
a:link, a:visited {
  color: #334488ff;
  text-decoration: none;
}
@media (max-width: 700px) {
  div {
    margin: 0 auto;
    width: auto;
  }
}
</style>
</head>
<body>
<div>
<h1>Example Domain</h1>
<p>This domain is for use in illustrative examples in documents. You may use this
domain in literature without prior coordination or asking for permission.</p>
<p><a href="https://www.iana.org/domains/example">More information...</a></p>
</div>
</body>
</html>
```

By specifying the CA file, curl validated the certificate chain:

The command returned the website content successfully, as the server certificate was signed by the trusted modelCA.crt.

In addition to curl, the openssl s_client tool was used to verify the certificate chain and server behavior. The following command validated the server certificate against the custom CA:

[illegible]

After testing, the MITM environment was dismantled as follows: The `www.example.com` ping was removed in the nano editor for the `.conf` file. Then, the DNS cache was flushed to apply the changes.

- For MITM Lab

The fake site configuration file (bank32_apache_ssl.conf) was initially located in the image_www directory. It was moved to the /etc/apache2/sites-available directory. After the move, the fake site was disabled using the a2dissite command. It was already disabled.

```

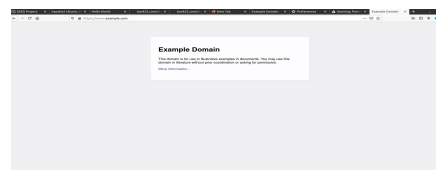
[12/07/24]seed@MN:~/.../image_www$ sudo mv bank32_apache_ssl.conf /etc/apache2/sites-available/bank32_ssl.conf
[12/07/24]seed@MN:~/.../image_www$ sudo a2dissite bank32_ssl.conf
Site bank32_ssl already disabled
[12/07/24]seed@MN:~/.../image_www$ sudo service apache2 restart
Job for apache2.service failed because the control process exited with error code.
See "systemctl status apache2.service" and "journalctl -xe" for details.

```

A ping test confirmed that `www.example.com` resolved to its legitimate IP address:

```
[12/07/24]seed@VM:~/.../image$ ping www.example.com
PING www.example.com (93.184.215.14) 56(84) bytes of data:
34 bytes from 93.184.215.14: (93.184.215.14): icmp_seq=1 ttl=55 time=43.2 ms
34 bytes from 93.184.215.14: (93.184.215.14): icmp_seq=2 ttl=55 time=16.5 ms
34 bytes from 93.184.215.14: (93.184.215.14): icmp_seq=3 ttl=55 time=17.1 ms
34 bytes from 93.184.215.14: (93.184.215.14): icmp_seq=4 ttl=55 time=16.7 ms
```

Finally, the browser was used to navigate to <https://www.example.com>. The fake site was no longer accessible, and the legitimate website loaded successfully.



I learned the importance of proper certificate validation and DNS configurations in preventing MITM attacks. Using curl and openssl provided insights into client behavior when connecting to HTTPS sites. The cleanup process ensured that the fake site was disabled and DNS spoofing entries were removed, allowing the legitimate `www.example.com` to be accessed without interference. These steps reinforced the need for robust PKI and DNS security practices.

III. CONCLUSION

The first few tasks focused on generating a custom Certificate Authority (CA) and issuing certificates for a fake website (`www.example.com`) to simulate how attackers could exploit weaknesses in certificate management. Using Apache as the web server, configurations were implemented to enable HTTPS for the fake website, and DNS spoofing was employed to redirect traffic to this malicious server.

Tools like curl and openssl were used to analyze the behavior of HTTPS clients when encountering valid, invalid, and mismatched certificates. These tools provided insights into the verification process and highlighted the importance of matching domain names and trusted certificate chains.

The tasks towards the end involved the cleanup and validation of the environment, including disabling the fake site, removing DNS spoofing entries, and restoring the legitimate domain's functionality. This process reinforced the importance of proper certificate management and secure DNS configurations in preventing and mitigating MITM attacks. All in all, Lab 4 reinforced foundational knowledge of HTTPS and PKI while demonstrating their critical role in secure communication.

REFERENCES

- [1] X. Yang, "CS 428/528/442/542 Lab requirements."
- [2] "SEED Project," *seedsecuritylabs.org*.
https://seedsecuritylabs.org/Labs_20.04/Crypto/Crypto_PKI/